

Churn Classifier with Pipelines + MLflow

Pipeline robuste de prédiction du churn sur IBM Telco Customer Churn, avec validation croisée, traçabilité complète MLflow, registre de modèles, tests automatisés, CI et conteneurisation.

0,8463	0,8411	8 / 8	0
ROC-AUC CV	ROC-AUC TEST	TESTS RÉUSSIS	VULNÉRABILITÉ

<https://github.com/ezzatsd/churn-classifier-mlflow>

AUTEUR	DATE	VERSION
ezzatsd	7 septembre 2026	1.0 - validée

01 Synthèse exécutive

L'objectif est de construire une solution reproductible capable d'estimer le risque de churn d'un client télécom à partir de données mixtes. Le résultat n'est pas un simple notebook: c'est un mini-produit MLOps exécutable par commandes, testé, configurable et traçable.

0,8463	0,8411	8 / 8	0
ROC-AUC CV	ROC-AUC TEST	TESTS RÉUSSIS	VULNÉRABILITÉ

Résultat principal

La régression logistique optimisée atteint un ROC-AUC de 0,8411 sur 1 409 clients jamais vus. La proximité avec le score de validation croisée (0,8463) indique une bonne stabilité et ne montre pas de surapprentissage marqué.

Livrables couverts

- Pipeline scikit-learn complète avec traitement numérique et catégoriel anti-fuite.
- Recherche d'hyperparamètres par GridSearchCV et StratifiedKFold à 5 plis.
- MLflow autolog + journalisation explicite des métriques, graphiques, prédictions et configuration.
- Model Registry: ChurnClassifier v1, alias staging et stage Staging.
- CLI, YAML, Makefile, Docker, GitHub Actions, 8 tests, audit de sécurité et model card.

DÉCISION

Le prototype est techniquement validé pour une démonstration. Une mise en production réelle exige un seuil métier, une calibration et un audit d'équité.

02 Données et préparation

Jeu de données

Le projet utilise le jeu public IBM Telco Customer Churn: 7 043 clients, 21 colonnes brutes et une cible binaire Churn. La classe positive représente environ 26,5 % des lignes, ce qui justifie un split stratifié et des métriques au-delà de l'accuracy.

Bloc	Variables	Traitement dans la pipeline
Numérique	tenure, MonthlyCharges, TotalCharges	Médiane → StandardScaler
Catégoriel	16 variables de contrat, service et profil	Mode → OneHotEncoder
Exclu	customerID	Identifiant sans signal généralisable
Cible	Churn (Yes / No)	Encodage binaire 1 / 0

Prévention de la fuite de données

Les 11 chaînes blanches de TotalCharges sont converties en valeurs manquantes, mais aucune statistique n'est calculée en amont. L'imputation, la standardisation et l'encodage sont des étapes internes au Pipeline; GridSearchCV les réajuste indépendamment dans chaque pli.

- Split holdout unique de 20 %, stratifié et déterministe (graine 42).
- Catégories inconnues tolérées à l'inférence avec `handle_unknown='ignore'`.
- Données brutes exclues de Git; script de téléchargement reproductible depuis IBM.
- SHA-256 observé: 16320c9c1ec72448...e3055e91; 7 043 lignes contrôlées.

03 Pipeline et sélection du modèle

DONNÉES BRUTES	SPLIT STRATIFIÉ	PIPELINE	MLFLOW
7 043 clients 19 variables utiles	80 % train 20 % test random_state=42	imputation → scaling / OHE → régression logistique	Runs → artefacts → registre → Staging

Choix du modèle

La régression logistique constitue un compromis robuste pour ce dataset tabulaire: probabilités exploitables, coût faible, forte lisibilité et intégration naturelle avec une représentation one-hot. Le code sait également construire une RandomForest via la configuration.

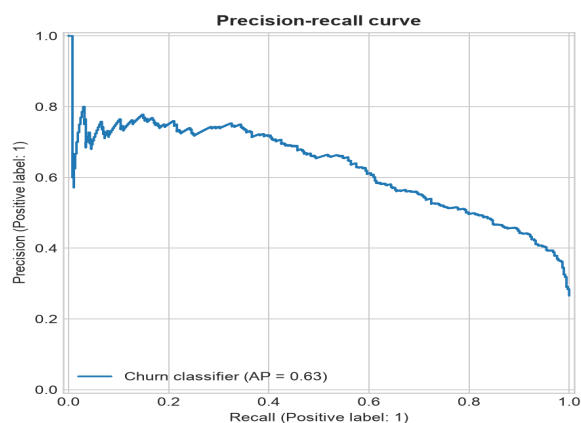
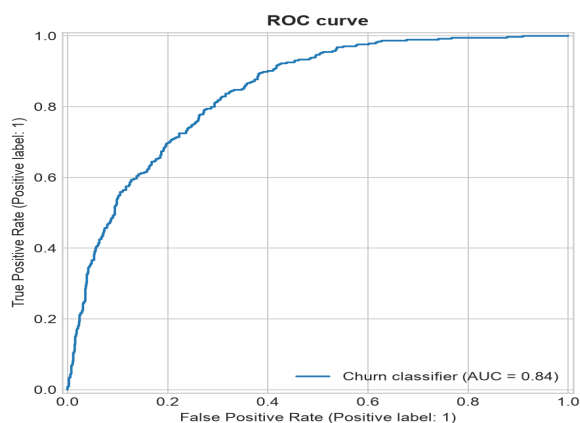
Élément	Configuration
Validation	StratifiedKFold, 5 plis, shuffle=True, random_state=42
Objectif	ROC-AUC
Grille	3 valeurs de C × 2 solvers × 2 class_weight = 12 candidats
Total	60 ajustements de validation croisée + refit final
Meilleur	C=10 · liblinear · l2 · class_weight=None

Pourquoi le ROC-AUC ?

Il évalue le classement des clients à risque sur tous les seuils et résiste mieux au déséquilibre que l'accuracy. L'average precision est ajoutée pour rendre visible la qualité sur la classe churn, plus rare. Le seuil de 0,50 reste une convention à adapter au coût métier.

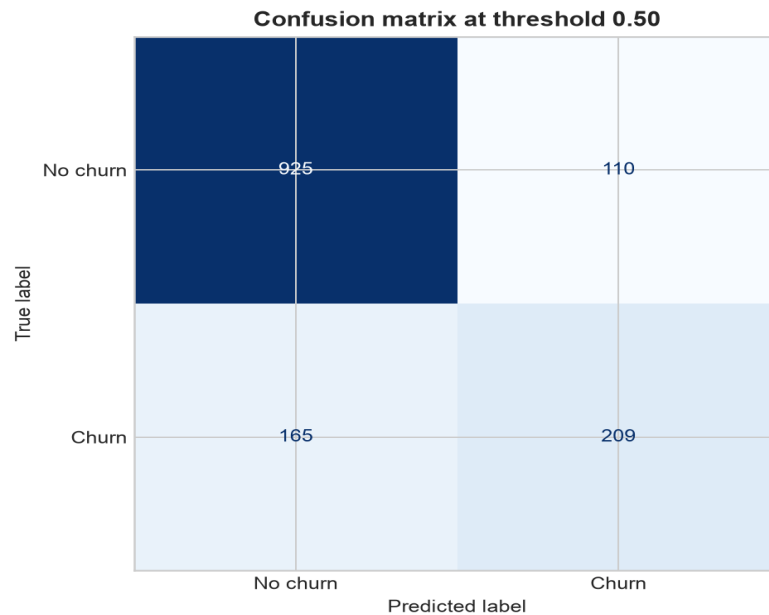
04 Résultats sur le holdout

Métrique	Validation croisée	Test final
ROC-AUC	0,8463	0,8411
Average precision	-	0,6281
Accuracy	-	0,8048
Balanced accuracy	-	0,7263
Précision	-	0,6552
Rappel	-	0,5588
F1-score	-	0,6032



Lecture: l'écart CV/test est de seulement 0,0052 point de ROC-AUC. L'average precision de 0,6281 est nettement supérieure au taux de base positif ($\sim 0,265$), ce qui confirme la valeur du classement pour prioriser des actions de rétention.

05 Analyse des erreurs



TN	FP	FN	TP	Erreurs
925	110	165	209	275 / 1 409

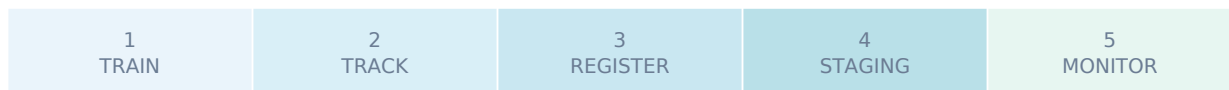
Le rappel de 0,5588 montre que 165 churners ne sont pas détectés au seuil 0,50. Dans une campagne où un faux négatif coûte davantage qu'un contact inutile, il faudrait abaisser le seuil et choisir le point optimal à partir d'une matrice de coûts. Le CSV de prédictions journalisé par MLflow permet précisément cette analyse.

06 Tracking MLflow et registre

Chaque exécution crée une trace auditable. L'autolog capture les candidats de GridSearchCV; le code journalise en plus les éléments métier et le holdout final dans un run parent.

Catégorie	Contenu tracé
Paramètres	split, seed, taille, features, 12 candidats, meilleur jeu
Métriques	CV best score + 7 métriques du test final
Artefacts	config YAML, ROC, PR, confusion, predictions, cv_results, résumé JSON
Modèle	pipeline complète, signature, exemple d'entrée, seuil en metadata
Registre	ChurnClassifier v1 · alias staging · stage Staging

Cycle du modèle



Le stockage local SQLite permet d'utiliser immédiatement l'interface MLflow. Les variables `MLFLOW_TRACKING_URI` et `MLFLOW_EXPERIMENT_NAME` rendent le même code compatible avec un serveur distant. Les alias modernes sont utilisés en complément des stages historiques dépréciés.

07 Industrialisation, sécurité et reproductibilité

```
churn-classifier-mlflow/
├─ configs/config.yaml
├─ src/{pipeline,train,evaluate,predict,utils}.py
├─ tests/ # 8 tests
├─ docs/model_card.md / SECURITY.md
├─ .github/{workflows/ci.yml,dependabot.yml}
├─ Makefile / Dockerfile
└─ requirements.txt
```

Contrôles automatisés

Contrôle	Résultat	Rôle
pytest	8 / 8	Pipeline, NaN, catégorie inconnue, modèle, split et métriques
Ruff	0 erreur	Imports, erreurs probables et cohérence Python 3.12
GitHub Actions	Configurée	Tests, audit, lint et build Docker à chaque push / pull request
pip-audit	0 vulnérabilité	Audit des dépendances à chaque exécution CI
Supply chain	Durcie	Actions par SHA + checksum SHA-256 des données
Batch inference	7 043 lignes	Validation du chargement du modèle et de predict_proba

Commandes reproductibles

```
make init
make data
make test && make lint && make audit
make train && make evaluate
make mlflow-ui
```

- Configuration externalisée en YAML et surcharge MLflow par variables d'environnement.
- Dockerfile Python 3.12 slim exécuté par un utilisateur non privilégié.
- Dependabot hebdomadaire pour Python, Docker et GitHub Actions.
- Modèle complet sérialisé: le consommateur n'a pas à reproduire le prétraitement.

08 Limites, gouvernance et conclusion

Limites connues

- Dataset pédagogique et ancien: aucune garantie de généralisation à une population actuelle.
- Seuil 0,50 non optimisé selon la valeur d'un client et le coût d'une action de rétention.
- Probabilités non calibrées; une calibration doit précéder leur usage financier.
- Équité non auditée par sous-groupe; les variables sensibles ne doivent pas piloter une décision automatisée.
- Pas de monitoring en ligne dans ce prototype; dérive des données et des performances à instrumenter.

Améliorations recommandées

Priorité	Action	Valeur attendue
P1	Optimiser le seuil par matrice de coûts	Aligner rappel et budget de campagne
P1	Calibration + courbe de fiabilité	Probabilités interprétables
P2	Comparer gradient boosting / RF	Mesurer le gain face à la baseline
P2	Monitoring et tests de dérive	Sécuriser la performance dans le temps

Conclusion

Le projet couvre l'intégralité du cycle demandé: ingestion reproductible, preprocessing anti-fuite, tuning, évaluation, suivi MLflow, registre, inférence et qualité logicielle. Le score test stable et la traçabilité complète en font un rendu solide, démontrable et extensible.

Dépôt GitHub

<https://github.com/ezzatsd/churn-classifier-mlflow>

Source des données: IBM, Telco Customer Churn on ICP4D. Documentation technique, model card, rapport et instructions d'exécution sont inclus dans le dépôt.